

Onyx — AppKit reference

Onyx · multi-process OS on Raspberry Pi 4

The reference of **AppKit** (`SD:/lib/appkit.so`, `user/Kits/appkit`): what it is for, how a program uses it, and every operation it exposes. The operations' part is made from the kit's headers by `tools/docgen/kitdocs.py` — the headers are the source. Overview of all the kits: [The Kits](#).

Contents

1. What it is
2. Using it
3. Index
4. `appkit/appkit.h`

What it is

AppKit is what makes a program run: its one link to the system. Every call a program makes to the kernel is a function of AppKit (`kapi_*`), and AppKit carries the small services every program needs — strings, the console, `.ini` files, the keyboard layout, the starting of programs. Only AppKit reads the kernel's table, so the kernel can change without a program being rebuilt.

Include	<code>#include "appkit/appkit.h"</code>
Link	nothing to link: the kernel binds AppKit to every program
Library	<code>SD:/lib/appkit.so</code> — 315 entries in its table (<code>user/Kits/appkit/appkit.abi</code> , append-only)
Sources	<code>user/Kits/appkit/</code>

Using it

AppKit is the program's link to the system: files, processes, time, sockets, the events' pump... (the `kapi_*` calls, listed in the Developer Guide; the windows are UIKit's since 2026-10-08: `uk_win_*`, §4), plus the small services every program needs: strings without a C library, the console, a reader of `.ini` files, the starting of other programs.

A console tool: read a file, print to the console.

```
#include "appkit/appkit.h"
```

```
int main (void)
{
    char buf[256];
    void *f = kapi_open ("SD:/etc/autostart");
    if (f == 0) { ax_putln ("no such file"); return 1; }
    int n = kapi_read (f, buf, sizeof buf - 1);
    kapi_close (f);
    buf[n > 0 ? n : 0] = '\0';
```

```

    ax_putln ("SD:/etc/autostart says:");
    ax_putln (buf);
    return 0;
}

```

Settings from an .ini file in the program's own folder:

```

if (app_ini_load ("config.ini") >= 0)
{
    const char *name = app_ini_get ("window", "title", "Untitled");
    int width      = app_ini_get_int ("window", "width", 640);
    for (int i = 0; i < app_ini_count (); i++)           // or every entry, in the
file's order
        ax_putln (app_ini_key (i));
}

```

Strings (no C library needed):

```

char path[96]; int n = 0;
ax_strcat (path, sizeof path, &n, "SD:/docs/");           // never overflows, always
terminated
ax_strcat (path, sizeof path, &n, "notes.txt");
char num[12]; ax_itoa (42, num);
if (ax_streq (num, "42")) ax_putln (path);

```

Starting a program — an application by its name, or a file by whatever runs it (a .bas program by BASIC, a game by its emulator):

```

lx_launch ("tinypad", "SD:/docs/notes.txt");             // an app, with its arguments
lx_open ("SD:/games/tetris.gb", 0);                       // a file: by its runner

```

The volumes, USB sticks (kapi v93) — list them, eject a stick, format one:

```

struct kapi_volume v[16];
int n = kapi_vol_list (v, 16, KAPI_VOLS_ROOM);           // SD:, SD1:..., USB1:...,
USB1P1:..., RAM:
for (int i = 0; i < n && i < 16; i++)
    if ((v[i].flags & KAPI_VF_REMOVABLE) && v[i].state == KAPI_VST_MOUNTED)
        ax_putln (v[i].label);                           // a stick plugged in: "USB" +
its label

if (kapi_vol_eject ("USB1:", 0) == -KAPI_EBUSY)           // files open on it (they were
synced)
    kapi_vol_eject ("USB1:", KAPI_EJECT_FORCE);           // ... after asking the user

struct kapi_format f = { KAPI_FMT_EXFAT, 0, 0, "PHOTOS" };
kapi_vol_format ("USB1:", &f);                             // erases it; SD: is always
refused

```

Index

Everything the headers declare, in their order — the details are in each header's part below.

Name	What it does	Header
kapi_abi_version	The kernel's kapi version (KAPI_ABI_VERSION as the running kernel says	appkit.h

Name	What it does	Header
	it)	
<code>kapi_table_slot</code>	The address of a slot of the kernel's table (its index in 8-byte words)	<code>appkit.h</code>
<code>kapi_launch</code>	start the app n (SD:apps/.app/main) as a new process → 1, 0 failure	<code>appkit.h</code>
<code>kapi_list_tasks</code>	Every task, one line each " " (state R / S / B / N, kind a = an app or k = a kernel task), into b (s bytes) → how many.	<code>appkit.h</code>
<code>kapi_kill</code>	kill the app of that name → 1, 0 (not running, a kernel task, the caller)	<code>appkit.h</code>
<code>kapi_list_procs</code>	ps / kill by PID.	<code>appkit.h</code>
<code>kapi_kill_pid</code>	ps / kill by PID.	<code>appkit.h</code>
<code>kapi_proc_tree</code>	(v91) A process's tree	<code>appkit.h</code>
<code>kapi_set_keymap</code>	Keyboard layout	<code>appkit.h</code>
<code>kapi_get_keymap</code>	Keyboard layout	<code>appkit.h</code>
<code>kapi_exec</code>	Run an ELF at an absolute path with an argv string (fire-and-forget).	<code>appkit.h</code>
<code>kapi_screen_size</code>	Framebuffer size in pixels (for edge-pinned borderless windows).	<code>appkit.h</code>
<code>kapi_get_ticks</code>	Framebuffer size in pixels (for edge-pinned borderless windows).	<code>appkit.h</code>
<code>kapi_msleep</code>	Framebuffer size in pixels (for edge-pinned borderless windows).	<code>appkit.h</code>
<code>kapi_yield</code>	Framebuffer size in pixels (for edge-pinned borderless windows).	<code>appkit.h</code>
<code>kapi_exit</code>	Framebuffer size in pixels (for edge-pinned borderless windows).	<code>appkit.h</code>
<code>kapi_pump_events</code>	Run what is pending -- the kapi_post calls, then this program's events through their handlers -- and return (no wait).	<code>appkit.h</code>
<code>kapi_wait_for_exit</code>	pump the events	<code>appkit.h</code>

Name	What it does	Header
	(sleeping between them) until this window is asked to close	
<code>kapi_should_exit</code>	1 once this window was asked to close (its close box, <code>uk_win_app_toggle</code>), else 0	<code>appkit.h</code>
<code>kapi_draw_text_buf</code>	Draw kernel-font text into an arbitrary app-mapped 0x00RRGGBB buffer (e.g.	<code>appkit.h</code>
<code>kapi_font_width</code>	Draw kernel-font text into an arbitrary app-mapped 0x00RRGGBB buffer (e.g.	<code>appkit.h</code>
<code>kapi_font_height</code>	Draw kernel-font text into an arbitrary app-mapped 0x00RRGGBB buffer (e.g.	<code>appkit.h</code>
<code>kapi_meminfo</code>	Memory snapshot (KB)	<code>appkit.h</code>
<code>kapi_ram_detail</code>	ABI v33	<code>appkit.h</code>
<code>kapi_sbrk</code>	Per-process heap	<code>appkit.h</code>
<code>kapi_list_apps</code>	The names of the installed apps (the folders <code>SD:apps/.app</code>), one a line, into <code>b</code> (s bytes) → how many.	<code>appkit.h</code>
<code>kapi_get_datetime</code>	The local date and time (any pointer may be 0) → 1 a real date, 0 the clock is not set yet (the time since the boot).	<code>appkit.h</code>
<code>kapi_app_dir</code>	" <code>SD:apps/<this process's name>.app/</code> " into <code>b</code> (s bytes) → its length	<code>appkit.h</code>
<code>kapi_write</code>	<code>b</code> (the first 128 bytes of its <code>n</code>) to the kernel's log as a line of "app" → <code>n</code> , -1 a bad buffer (<code>fd</code> is not used).	<code>appkit.h</code>
<code>kapi_open</code>	Open a file to read it (on the card, the RAM volume or a provider's path, relative to the working directory) → its	<code>appkit.h</code>

Name	What it does	Header
	handle, 0 failure.	
<code>kapi_read</code>	up to n bytes from the file's position (advanced) → the bytes read (0: the end), -1 error	<code>appkit.h</code>
<code>kapi_fsize</code>	the file's size in bytes (0xFFFFFFFF: over 4 GB, see <code>kapi_fsize64</code>); 0 for a bad handle	<code>appkit.h</code>
<code>kapi_close</code>	close a file opened with <code>kapi_open</code>	<code>appkit.h</code>
<code>kapi_save_file</code>	<code>save_file</code>	<code>appkit.h</code>
<code>kapi_chdir</code>	Working directory	<code>appkit.h</code>
<code>kapi_getcwd</code>	Working directory	<code>appkit.h</code>
<code>kapi_opendir</code>	open a folder to list it → its handle, 0 failure (not a folder)	<code>appkit.h</code>
<code>kapi_readdir</code>	the next entry (name, size, is_dir) into *e → 1, 0 at the end / on error	<code>appkit.h</code>
<code>kapi_closedir</code>	close a folder opened with <code>kapi_opendir</code>	<code>appkit.h</code>
<code>kapi_mkdir</code>	<code>mkdir</code> / <code>remove</code> / <code>rename</code>	<code>appkit.h</code>
<code>kapi_remove</code>	<code>mkdir</code> / <code>remove</code> / <code>rename</code>	<code>appkit.h</code>
<code>kapi_rename</code>	<code>mkdir</code> / <code>remove</code> / <code>rename</code>	<code>appkit.h</code>
<code>kapi_pipe</code>	a new pipe (a FIFO in memory) → its stream handle, 0 failure	<code>appkit.h</code>
<code>kapi_file_in</code>	a file as a stream to read → its stream handle, 0 failure	<code>appkit.h</code>
<code>kapi_file_out</code>	A file as a stream to write, created / emptied or (append != 0) written at its end → its stream handle, 0 failure.	<code>appkit.h</code>
<code>kapi_stream_read</code>	Up to n bytes from a stream (a pipe waits for its writer) → the bytes read, 0 the end / a bad handle.	<code>appkit.h</code>
<code>kapi_stream_read_nb</code>	Up to n bytes from a	<code>appkit.h</code>

Name	What it does	Header
	stream without waiting → > 0 the bytes read, 0 the end / a bad handle, -1 nothing yet.	
<code>kapi_stream_write</code>	n bytes to a stream (a full pipe waits for its reader) → the bytes written, -1 error.	<code>appkit.h</code>
<code>kapi_stream_close</code>	close a stream handle (its reference to the stream dropped)	<code>appkit.h</code>
<code>kapi_stream_eof</code>	tell the stream's readers it has ended (the writer is done); the handle stays open	<code>appkit.h</code>
<code>kapi_proc_done</code>	1 if the process of <code>kapi_spawn</code> has finished (a bad handle too), 0 if it runs (the handle stays, <code>kapi_wait</code> closes it).	<code>appkit.h</code>
<code>kapi_stdin_read</code>	up to n bytes from this process's standard input → the bytes read, 0 the end / none	<code>appkit.h</code>
<code>kapi_stdout_write</code>	n bytes to this process's standard output (it has none = to the kernel's log, 128 bytes at most) → the bytes written, -1 error.	<code>appkit.h</code>
<code>kapi_spawn</code>	Start the program at path with the arguments args, its standard input / output the stream handles in / out (0 = none) → its process handle (<code>kapi_wait</code> , <code>kapi_pro</code>)	<code>appkit.h</code>
<code>kapi_wait</code>	wait for a spawned process's end → its exit status (the handle closed), -1 not a process handle	<code>appkit.h</code>
<code>kapi_get_args</code>	this process's argument string into b (n bytes) → its length	<code>appkit.h</code>
<code>kapi_stdin</code>	This task's own	<code>appkit.h</code>

Name	What it does	Header
	stdin/stdout stream handles (for a shell wiring children).	
kapi_stdout	This task's own stdin/stdout stream handles (for a shell wiring children).	appkit.h
kapi_klog_read	Read the next kernel log event (real-time tee).	appkit.h
kapi_set_verbose	Verbose kernel logging	appkit.h
kapi_get_verbose	Verbose kernel logging	appkit.h
kapi_net_status	TCP/IP sockets over WLAN (ABI v21).	appkit.h
kapi_tcp_connect	TCP/IP sockets over WLAN (ABI v21).	appkit.h
kapi_tcp_send	TCP/IP sockets over WLAN (ABI v21).	appkit.h
kapi_tcp_recv	TCP/IP sockets over WLAN (ABI v21).	appkit.h
kapi_tcp_close	TCP/IP sockets over WLAN (ABI v21).	appkit.h
kapi_tcp_listen	TCP server side (ABI v37).	appkit.h
kapi_tcp_accept	TCP server side (ABI v37).	appkit.h
kapi_screen_grab	Remote screen (ABI v38).	appkit.h
kapi_inject_pointer	Remote screen (ABI v38).	appkit.h
kapi_inject_key	Remote screen (ABI v38).	appkit.h
kapi_ipc_register	Named IPC services (ABI v40)	appkit.h
kapi_ipc_lookup	Named IPC services (ABI v40)	appkit.h
kapi_clipboard_set	The clipboard replaced by the n bytes of d (64 KB at most, n 0 empties it) of the type CLIP_* → the bytes kept (0 too for a bad pointer, the clipboard then as	appkit.h
kapi_clipboard_get	Up to cap bytes of the clipboard into b, its type and its serial (which	appkit.h

Name	What it does	Header
	changes at every set) → the content's whole length, 0 empty.	
<code>kapi_shutdown</code>	end the session: the card unmounted, then halt (SHUTDOWN_HALT) or restart; does not return	<code>appkit.h</code>
<code>kapi_fullscreen_begin</code>	Full-screen apps (ABI v41)	<code>appkit.h</code>
<code>kapi_present_fb</code>	Full-screen apps (ABI v41)	<code>appkit.h</code>
<code>kapi_fullscreen_end</code>	Full-screen apps (ABI v41)	<code>appkit.h</code>
<code>kapi_get_modifiers</code>	The keyboard's modifiers (ABI v42)	<code>appkit.h</code>
<code>kapi_inject_modifiers</code>	The keyboard's modifiers (ABI v42)	<code>appkit.h</code>
<code>kapi_net_ping</code>	Network tools (ABI v43).	<code>appkit.h</code>
<code>kapi_net_resolve</code>	Network tools (ABI v43).	<code>appkit.h</code>
<code>kapi_net_info</code>	Network tools (ABI v43).	<code>appkit.h</code>
<code>kapi_vfs_register</code>	this process serves the paths starting with prefix → 1, 0 (taken by another, no room)	<code>appkit.h</code>
<code>kapi_vfs_next</code>	The next request for this provider into *req → 1, 0 none (with blocking != 0, after waiting up to 0.5 s for one).	<code>appkit.h</code>
<code>kapi_vfs_req_data</code>	Up to cap bytes of the request id's payload (VFS_OP_SAVE's data) from offset into buf → the bytes copied, 0 none.	<code>appkit.h</code>
<code>kapi_vfs_reply</code>	Answer the request id with its status and len bytes of data (0 = none), its caller woken → 1, 0 (no such request, bad data).	<code>appkit.h</code>
<code>kapi_wlan_scan</code>	Wi-Fi scan (ABI v45)	<code>appkit.h</code>

Name	What it does	Header
<code>kapi_sound_acquire</code>	Sound (ABI v46), on the 3.5 mm jack.	<code>appkit.h</code>
<code>kapi_sound_release</code>	Sound (ABI v46), on the 3.5 mm jack.	<code>appkit.h</code>
<code>kapi_sound_start</code>	(RETIRED 2026-10-05)	<code>appkit.h</code>
<code>kapi_sound_stop</code>	(RETIRED 2026-10-05)	<code>appkit.h</code>
<code>kapi_sound_write</code>	(RETIRED 2026-10-05)	<code>appkit.h</code>
<code>kapi_sound_status</code>	(RETIRED 2026-10-05)	<code>appkit.h</code>
<code>kapi_sound_instrument</code>	FM (ABI v47)	<code>appkit.h</code>
<code>kapi_key_held</code>	Held keys (ABI v48), for games (key events only report presses)	<code>appkit.h</code>
<code>kapi_inject_key_held</code>	Held keys (ABI v48), for games (key events only report presses)	<code>appkit.h</code>
<code>kapi_exec_as</code>	Run a program under another process name (ABI v49)	<code>appkit.h</code>
<code>kapi_pad_state</code>	USB gamepads (v50)	<code>appkit.h</code>
<code>kapi_core_acquire</code>	App cores (v51)	<code>appkit.h</code>
<code>kapi_core_run</code>	App cores (v51)	<code>appkit.h</code>
<code>kapi_core_state</code>	App cores (v51)	<code>appkit.h</code>
<code>kapi_core_release</code>	App cores (v51)	<code>appkit.h</code>
<code>kapi_gpu_info</code>	The V3D GPU (v52)	<code>appkit.h</code>
<code>kapi_gpu_draw</code>	The V3D GPU (v52)	<code>appkit.h</code>
<code>kapi_gpu_texture</code>	The GPU's full pipeline (v53)	<code>appkit.h</code>
<code>kapi_gpu_render</code>	The GPU's full pipeline (v53)	<code>appkit.h</code>
<code>kapi_fullscreen_direct</code>	Full screen straight into the displayed framebuffer (v55)	<code>appkit.h</code>
<code>kapi_seek</code>	The read position of an opened file (v57)	<code>appkit.h</code>
<code>kapi_code_alloc</code>	Writable + executable memory for generated code, a JIT (v58)	<code>appkit.h</code>
<code>kapi_fsize64</code>	A file's whole size (v59)	<code>appkit.h</code>
<code>kapi_vol_info</code>	(v71) <code>vol_info</code>	<code>appkit.h</code>
<code>kapi_vol_list</code>	(v93) The volumes	<code>appkit.h</code>
<code>kapi_vol_eject</code>	<code>vol_eject</code>	<code>appkit.h</code>
<code>kapi_vol_mount</code>	<code>vol_mount</code>	<code>appkit.h</code>
<code>kapi_vol_format</code>	<code>vol_format</code>	<code>appkit.h</code>

Name	What it does	Header
<code>kapi_pop_event</code>	(v73) The event pump's kernel half -- what <code>kapi_pump_events</code> does, step by step, for a pump of the app's own (a protected app's table runs its pump that way, ker	<code>appkit.h</code>
<code>kapi_event_mods</code>	(v73) The event pump's kernel half -- what <code>kapi_pump_events</code> does, step by step, for a pump of the app's own (a protected app's table runs its pump that way, ker	<code>appkit.h</code>
<code>kapi_pop_post</code>	(v73) The event pump's kernel half -- what <code>kapi_pump_events</code> does, step by step, for a pump of the app's own (a protected app's table runs its pump that way, ker	<code>appkit.h</code>
<code>kapi_pump_sleep</code>	(v73) The event pump's kernel half -- what <code>kapi_pump_events</code> does, step by step, for a pump of the app's own (a protected app's table runs its pump that way, ker	<code>appkit.h</code>
<code>kapi_proc_stats</code>	(v74) A process's system calls (pid 0	<code>appkit.h</code>
<code>kapi_vm_map</code>	(v75) The POSIX layer's kernel half (docs/POSIX-PLAN.md	<code>appkit.h</code>
<code>kapi_vm_unmap</code>	(v75) The POSIX layer's kernel half (docs/POSIX-PLAN.md	<code>appkit.h</code>
<code>kapi_vm_protect</code>	(v75) The POSIX layer's kernel half (docs/POSIX-PLAN.md	<code>appkit.h</code>
<code>kapi_vm_advise</code>	(v75) The POSIX layer's kernel half (docs/POSIX-PLAN.md	<code>appkit.h</code>
<code>kapi_vm_query</code>	(v75) The POSIX layer's	<code>appkit.h</code>

Name	What it does	Header
	kernel half (docs/POSIX-PLAN.md)	
<code>kapi_vm_stats</code>	(v75) The POSIX layer's kernel half (docs/POSIX-PLAN.md)	<code>appkit.h</code>
<code>kapi_thread_create_ex</code>	(v75) The POSIX layer's kernel half (docs/POSIX-PLAN.md)	<code>appkit.h</code>
<code>kapi_thread_info</code>	(v75) The POSIX layer's kernel half (docs/POSIX-PLAN.md)	<code>appkit.h</code>
<code>kapi_file_open</code>	v75 WP-FILE/PROC	<code>appkit.h</code>
<code>kapi_file_read</code>	v75 WP-FILE/PROC	<code>appkit.h</code>
<code>kapi_file_write</code>	v75 WP-FILE/PROC	<code>appkit.h</code>
<code>kapi_file_seek</code>	v75 WP-FILE/PROC	<code>appkit.h</code>
<code>kapi_file_truncate</code>	v75 WP-FILE/PROC	<code>appkit.h</code>
<code>kapi_file_sync</code>	v75 WP-FILE/PROC	<code>appkit.h</code>
<code>kapi_file_stat</code>	v75 WP-FILE/PROC	<code>appkit.h</code>
<code>kapi_file_close</code>	v75 WP-FILE/PROC	<code>appkit.h</code>
<code>kapi_path_stat</code>	v75 WP-FILE/PROC	<code>appkit.h</code>
<code>kapi_path_unlink</code>	v75 WP-FILE/PROC	<code>appkit.h</code>
<code>kapi_path_mkdir</code>	v75 WP-FILE/PROC	<code>appkit.h</code>
<code>kapi_path_rename</code>	v75 WP-FILE/PROC	<code>appkit.h</code>
<code>kapi_path_utime</code>	v75 WP-FILE/PROC	<code>appkit.h</code>
<code>kapi_dir_read</code>	v75 WP-FILE/PROC	<code>appkit.h</code>
<code>kapi_stream_write_nb</code>	v75 WP-FILE/PROC	<code>appkit.h</code>
<code>kapi_spawn_ex</code>	v75 WP-FILE/PROC	<code>appkit.h</code>
<code>kapi_proc_wait</code>	v75 WP-FILE/PROC	<code>appkit.h</code>
<code>kapi_get_argv</code>	v75 WP-FILE/PROC	<code>appkit.h</code>
<code>kapi_get_env</code>	v75 WP-FILE/PROC	<code>appkit.h</code>
<code>kapi_getpid</code>	v75 WP-FILE/PROC	<code>appkit.h</code>
<code>kapi_clock_info</code>	v75 WP-FILE/PROC	<code>appkit.h</code>
<code>kapi_sleep_us</code>	v75 WP-FILE/PROC	<code>appkit.h</code>
<code>kapi_sock_open</code>	v75 WP-NET	<code>appkit.h</code>
<code>kapi_sock_connect</code>	v75 WP-NET	<code>appkit.h</code>
<code>kapi_sock_bind</code>	v75 WP-NET	<code>appkit.h</code>
<code>kapi_sock_listen</code>	v75 WP-NET	<code>appkit.h</code>
<code>kapi_sock_accept</code>	v75 WP-NET	<code>appkit.h</code>
<code>kapi_sock_send</code>	v75 WP-NET	<code>appkit.h</code>
<code>kapi_sock_recv</code>	v75 WP-NET	<code>appkit.h</code>
<code>kapi_sock_shutdown</code>	v75 WP-NET	<code>appkit.h</code>

Name	What it does	Header
kapi_sock_close	v75 WP-NET	appkit.h
kapi_sock_getopt	v75 WP-NET	appkit.h
kapi_sock_setopt	v75 WP-NET	appkit.h
kapi_sock_name	v75 WP-NET	appkit.h
kapi_poll	v75 WP-NET	appkit.h
kapi_sock_pair	(v76, WP-IPC	appkit.h
kapi_sock_sendmsg	(v76, WP-IPC	appkit.h
kapi_sock_recvmsg	(v76, WP-IPC	appkit.h
kapi_shm_create	(v76, WP-IPC	appkit.h
kapi_shm_open	(v76, WP-IPC	appkit.h
kapi_shm_unlink	(v76, WP-IPC	appkit.h
kapi_shm_ctl	(v76, WP-IPC	appkit.h
kapi_shm_map	(v76, WP-IPC	appkit.h
kapi_handle_close	(v76, WP-IPC	appkit.h
kapi_spawn_ex2	(v76, WP-IPC	appkit.h
kapi_get_handles	(v76, WP-IPC	appkit.h
kapi_image_preload	(v77) Program images (docs/02 section 7)	appkit.h
kapi_image_unload	(v77) Program images (docs/02 section 7)	appkit.h
kapi_image_list	(v77) Program images (docs/02 section 7)	appkit.h
kapi_kernel_info	(v79) What the running kernel is	appkit.h
kapi_cpu_stats	(v80) The cores	appkit.h
kapi_net_stats	(v80) The bytes pid's sockets received and sent, its open sockets (pid 0	appkit.h
kapi_lib_open	(v83) A shared library (docs/SHARED-LIBS- PLAN.md)	appkit.h
kapi_lib_open_as	(v97) The graphics server's own library under another name (docs/POCKETUI- TECH-STUDY.md section 3)	appkit.h
kapi_sound_output	(v84) The sound's output	appkit.h
kapi_sound_clients	(v85) the sound's mixer	appkit.h
kapi_sound_client_vol	(v85) the sound's mixer	appkit.h

Name	What it does	Header
ume		
kapi_is_protected	(v73) Is this process protected (ELO, kern/el0.h)? Its table's memcpy is then user code, next to the table, instead of the kernel's.	appkit.h
kapi_sound_volume	the master volume 0..10 and mute (-1)	appkit.h
kapi_wlan_reconnect	wpa_supplicant.conf read again + DHCP again, no reboot (-1)	appkit.h
kapi_gpu_program	v61	appkit.h
kapi_gpu_render2	v61	appkit.h
kapi_gpu_render3	(v62) gpu_render3	appkit.h
kapi_gpu_vbuf	(v63) gpu_vbuf	appkit.h
kapi_gpu_texture_rect	(v70) gpu_texture_rect	appkit.h
kapi_screen_set	(v66) screen_set	appkit.h
kapi_thread_create	(v67) Threads	appkit.h
kapi_thread_exit	(v67) Threads	appkit.h
kapi_thread_join	(v67) Threads	appkit.h
kapi_thread_self	(v67) Threads	appkit.h
kapi_mutex_create	Synchronisation objects (handles > 0)	appkit.h
kapi_mutex_lock	Synchronisation objects (handles > 0)	appkit.h
kapi_mutex_unlock	Synchronisation objects (handles > 0)	appkit.h
kapi_event_create	Synchronisation objects (handles > 0)	appkit.h
kapi_event_set	Synchronisation objects (handles > 0)	appkit.h
kapi_event_reset	Synchronisation objects (handles > 0)	appkit.h
kapi_event_wait	Synchronisation objects (handles > 0)	appkit.h
kapi_barrier_create	Synchronisation objects (handles > 0)	appkit.h
kapi_barrier_wait	Synchronisation objects (handles > 0)	appkit.h
kapi_sync_close	Synchronisation objects (handles > 0)	appkit.h
kapi_post	Synchronisation objects	appkit.h

Name	What it does	Header
	(handles > 0	
<code>kapi_pump_wait</code>	Synchronisation objects (handles > 0	<code>appkit.h</code>
<code>kapi_lock</code>	(on an app core -- kapi_core_run's code, which makes no kapi call -- it spins instead)	<code>appkit.h</code>
<code>kapi_unlock</code>	(on an app core -- kapi_core_run's code, which makes no kapi call -- it spins instead)	<code>appkit.h</code>
<code>kapi_wait_word</code>	(v68) A futex.	<code>appkit.h</code>
<code>kapi_wake_word</code>	(v68) A futex.	<code>appkit.h</code>
<code>kapi_thread_priority</code>	(v68) A futex.	<code>appkit.h</code>
<code>kapi_midi_read</code>	(v68) USB MIDI input	<code>appkit.h</code>
<code>kapi_midi_devices</code>	(v68) USB MIDI input	<code>appkit.h</code>
<code>kapi_screen_native</code>	(v69) kapi_screen_native (&w, &h)	<code>appkit.h</code>
<code>kapi_set_timezone</code>	(v69) kapi_screen_native (&w, &h)	<code>appkit.h</code>
<code>kapi_clock_us</code>	The kernel's microsecond clock (CTimer::GetClockTicks	<code>appkit.h</code>
<code>kapi_sound_config</code>	(v68) Low-latency sound, for the sound owner (kapi_sound_acquire).	<code>appkit.h</code>
<code>kapi_sound_map</code>	(v68) Low-latency sound, for the sound owner (kapi_sound_acquire).	<code>appkit.h</code>
<code>kapi_sound_ring_free</code>	Frames the ring can take now.	<code>appkit.h</code>
<code>kapi_sound_ring_write</code>	Copy up to n s16 stereo frames into the ring → the frames taken (no kapi call	<code>appkit.h</code>
<code>kapi_reboot</code>	Reboot the machine (ABI v25).	<code>appkit.h</code>
<code>kapi_kbd_ready</code>	1 if a USB keyboard is attached & ready, else 0 (ABI v26).	<code>appkit.h</code>
<code>kapi_set_keymap_data</code>	Load a keyboard layout	<code>appkit.h</code>

Name	What it does	Header
	from a .kmap blob (ABI v27)	
<code>kapi_random</code>	Hardware RNG (ABI v30)	<code>appkit.h</code>
<code>kapi_surface_create</code>	Shell surfaces (ABI v35)	<code>appkit.h</code>
<code>kapi_surface_map</code>	Shell surfaces (ABI v35)	<code>appkit.h</code>
<code>kapi_surface_size</code>	Shell surfaces (ABI v35)	<code>appkit.h</code>
<code>kapi_surface_present</code>	Shell surfaces (ABI v35)	<code>appkit.h</code>
<code>kapi_surface_destroy</code>	Shell surfaces (ABI v35)	<code>appkit.h</code>
<code>kapi_register_shell</code>	Activity-shell IPC (ABI v35)	<code>appkit.h</code>
<code>kapi_shell_request</code>	Activity-shell IPC (ABI v35)	<code>appkit.h</code>
<code>kapi_mailbox_send</code>	Activity-shell IPC (ABI v35)	<code>appkit.h</code>
<code>kapi_mailbox_recv</code>	Activity-shell IPC (ABI v35)	<code>appkit.h</code>
<code>kapi_ws_ctl</code>	(v89) The graphics server's own door to the kernel (Elegant, SD:/bin/elegant)	<code>appkit.h</code>
<code>kapi_memset</code>	Memory primitives (ABI v36)	<code>appkit.h</code>
<code>kapi_memcpy</code>	Memory primitives (ABI v36)	<code>appkit.h</code>
<code>kapi_memmove</code>	Memory primitives (ABI v36)	<code>appkit.h</code>
<code>get_ticks</code>	Friendly aliases used by the demos (create_window, present)	<code>appkit.h</code>
<code>msleep</code>	Friendly aliases used by the demos (create_window, present)	<code>appkit.h</code>
<code>pump_events</code>	Friendly aliases used by the demos (create_window, present)	<code>appkit.h</code>
<code>should_exit</code>	Friendly aliases used by the demos (create_window, present)	<code>appkit.h</code>
<code>ax_strcat</code>	Strings.	<code>appkit.h</code>
<code>ax_app_path</code>	Strings.	<code>appkit.h</code>

Name	What it does	Header
<code>ax_streq</code>	Strings.	<code>appkit.h</code>
<code>ax_strlen</code>	Strings.	<code>appkit.h</code>
<code>ax_itoa</code>	Strings.	<code>appkit.h</code>
<code>ax_fmt2</code>	Strings.	<code>appkit.h</code>
<code>ax_puts</code>	The console	<code>appkit.h</code>
<code>ax_putln</code>	The console	<code>appkit.h</code>
<code>app_ini_load_path</code>	load the .ini file at path (replacing the one loaded) → its number of entries, -1 not opened	<code>appkit.h</code>
<code>app_ini_load</code>	the same for <the program's own folder>/filename (<code>kapi_app_dir</code>)	<code>appkit.h</code>
<code>app_ini_get</code>	The value of key in section (0 or "" = the keys before any [section]) of the loaded file, else def.	<code>appkit.h</code>
<code>app_ini_get_int</code>	The value of key in section as a decimal integer (a sign allowed), def when the key is absent or has no digit.	<code>appkit.h</code>
<code>app_ini_count</code>	the number of entries (key=value lines) of the loaded file	<code>appkit.h</code>
<code>app_ini_section</code>	the section of the entry i (0-based, in the file's order); "" out of range	<code>appkit.h</code>
<code>app_ini_key</code>	the key of the entry i; "" out of range	<code>appkit.h</code>
<code>app_ini_value</code>	the value of the entry i; "" out of range	<code>appkit.h</code>
<code>ax_load_keymap</code>	The keyboard layout ("FR", "BE"...)	<code>appkit.h</code>
<code>lx_low</code>	c in lower case ('A'..'Z' only)	<code>appkit.h</code>
<code>lx_len</code>	the length of s (0 for a null pointer)	<code>appkit.h</code>
<code>lx_cat</code>	s appended to d at *n (advanced), never past cap, NUL-terminated	<code>appkit.h</code>
<code>lx_exists</code>	1 if the file at path can be opened (<code>kapi_open</code>),	<code>appkit.h</code>

Name	What it does	Header
	else 0	
<code>lx_entry</code>	The i-th "ext = program" line of runners.ini (0-based)	<code>appkit.h</code>
<code>lx_lists_ext</code>	Does the value of an app.txt's "games" / "opens" (the extensions after each "System:", or all the words of "opens") hold ext?	<code>appkit.h</code>
<code>lx_app_for</code>	The app (SD:/apps/.app/main) whose app.txt opens files of this extension	<code>appkit.h</code>
<code>lx_runner</code>	The runner of a file, by its extension (runners.ini, else the built-in list)	<code>appkit.h</code>
<code>lx_cmdline</code>	The runner's command line	<code>appkit.h</code>
<code>lx_open_as</code>	A program file	<code>appkit.h</code>
<code>lx_open</code>	A program file	<code>appkit.h</code>
<code>lx_launch_dir</code>	An app bundle by folder path (".../x.app")	<code>appkit.h</code>
<code>lx_launch</code>	An app by name (SD:apps/.app), with arguments (0 / ""	<code>appkit.h</code>

appkit/appkit.h

appkit.h -- AppKit: what a program includes to talk to the system.

A program calls the functions declared here (`kapi_*`: the names of AppKit's table, kept as they were); they are AppKit's (SD:/lib/appkit.so, user/Kits/appkit), which the kernel binds to every program. A program never talks to the kernel itself. Their bodies are in `appkit_calls.inc`, beside this file. The structures and constants shared with the kernel come from `<kern/kapi_abi.h>`.

(This header was `user/kapi.h` until 2026-10-05. There is no `kapi.h` any more: what talks to the kernel is in this folder only.)

how a program reaches the kernel: AppKit (2026-10-05)

The `kapi_*` functions declared below (`KAPI_FN`) are AppKit's (SD:/lib/appkit.so, user/Kits/appkit: the ONE interface between the programs and the kernel -- loaded by the kernel and bound to every program with no call of theirs). A program calls them BY NAME, through the import stubs linked into it (`lib/appkit_stubs.o`). THIS HEADER ONLY DECLARES THEM: their bodies -- the calls through the kernel's table -- are in `appkit_calls.inc` (beside this file), compiled into AppKit (`appkit/appkit.c`) and nowhere else. So the kernel's table can be restructured -- entries moved, removed, merged -- by adapting that file and rebuilding AppKit alone: no program changes.

The one exception: with KAPI_INLINE (the tests of the table itself: el0test, faulttest), or in a PC build (the simulator's stand-in kernel has a table of its own and no AppKit), the bodies are compiled inline into the program -- the include at this header's end.

What stays inline here makes no call to the kernel (the spin locks, kapi_clock_us, the sound ring's writer): an app core may use it.

```
#define KAPI_C extern "C"
#define KAPI_C
#define KAPI_FN static inline
#define KAPI_FN KAPI_C
```

The kernel's kapi version (KAPI_ABI_VERSION as the running kernel says it): what a program tests before a call that came late (it was the first word of the kernel's table).

```
unsigned kapi_abi_version (void);
```

The address of a slot of the kernel's table (its index in 8-byte words): for the tests of the table itself. Nothing else needs it.

```
unsigned long long kapi_table_slot (unsigned slot);
```

Window creation flags (must match kern/gui/window.h).

```
#define WIN_FLAG_BORDERLESS (1u << 0) // no title bar / border / close box
#define WIN_FLAG_BACKMOST (1u << 1) // pinned to the bottom of the z-order (shell desktop)
#define WIN_FLAG_TOPMOST (1u << 2) // pinned to the top, never active (the menu bar)
#define WIN_FLAG_TRANSPARENT (1u << 3) // magenta (0xFF00FF) client pixels are see-through
#define WIN_FLAG_SYSTEM (1u << 4) // system component: not listed as an open app (panel taskbar)
#define WIN_FLAG_ALPHA (1u << 5) // (v64, borderless) the pixels' top byte is a transparency
```

(0 opaque .. 255 see-through; clicks there go below)

```
#define WIN_FLAG_FIXED (1u << 6) // (v69) not movable, no title buttons, kept centred
```

Event kinds (must match kern/gui/window.h).

```
#define GUI_EVENT_CLICK 1
#define GUI_EVENT_CHECK_CHANGED 2
#define GUI_EVENT_TEXT_CHANGED 3
#define GUI_EVENT_VALUE_CHANGED 4
#define GUI_EVENT_KEY 5 // key pressed; value = char or KEY_* code
#define GUI_EVENT_CANVAS_CLICK 6 // client-area press; value = (buttons<<32)|(x<<16)|y
#define GUI_EVENT_CANVAS_MOTION 7 // drag (button held) over the client area; same value
```

buttons: bit0 left, bit1 right Full pointer stream (ABI v22, opt-in via uk_win_on_pointer) for app-side widget toolkits (UIKit.h). value packs (wheel<<48)|(changed<<40)|(buttons<<32)|(x<<16)|y, all client-relative; decode with the GUI_PTR_* macros below.

```
#define GUI_EVENT_PTR_MOVE 8 // cursor moved
#define GUI_EVENT_PTR_DOWN 9 // a button went down (changed = which: 1/2/4)
#define GUI_EVENT_PTR_UP 10 // a button went up (changed = which)
#define GUI_EVENT_PTR_ENTER 11 // cursor entered the client area
```

```
#define GUI_EVENT_PTR_LEAVE 12 // cursor left the client area
#define GUI_EVENT_PTR_WHEEL 13 // scroll wheel turned (GUI_PTR_WHEEL = signed notch
delta)
#define GUI_EVENT_MENU 14 // menu-bar command chosen (value = item id, ABI v39)
#define MENU_QUIT (-1) // uk_win_menu_command id: close the active app
```

Drag & drop (ABI v42) -- to the pointer handler (see uk_win_drag_begin):

```
#define GUI_EVENT_DROP 15 // dropped on us: GUI_PTR_X/Y + GUI_DND_FLAGS;
uk_win_drag_data
#define GUI_EVENT_DRAG_OVER 16 // a drag hovers us (GUI_DND_FLAGS & DND_F_LEAVE: it
left)
#define GUI_EVENT_DRAG_DONE 17 // to the source: GUI_DND_PID (0 = none) +
GUI_DND_FLAGS
#define GUI_EVENT_DISPLAY_RESIZE 19 // (v66) the screen's size changed: GUI_DISPLAY_W
/ _H (value)
#define GUI_DISPLAY_W(v) ((int) (((v) >> 16) & 0xFFFF))
#define GUI_DISPLAY_H(v) ((int) ((v) & 0xFFFF))
#define GUI_EVENT_WINCTL 18 // (v64) a title button: value KAPI_FRAME_MENU (the
window menu)
```

or KAPI_FRAME_MAXIMISE (also a double click on the title)

```
#define GUI_DND_FLAGS(v) ((unsigned) (((unsigned long long) (v) >> 32) & 0xFF))
#define GUI_DND_PID(v) ((int) (((unsigned long long) (v) & 0xFFFFFFFF)))
#define DND_F_COPY 1 // Ctrl held: copy instead of move
#define DND_F_CANCEL 2 // DRAG_DONE: cancelled (Esc)
#define DND_F_DESKTOP 4 // DRAG_DONE: dropped on the desktop / no window
#define DND_F_LEAVE 8 // DRAG_OVER: the drag left this window
#define DND_TEXT 1 // payload types (= CLIP_TEXT / CLIP_FILES)
#define DND_FILES 2
#define MOD_CTRL 1 // kapi_get_modifiers
#define MOD_SHIFT 2
#define MOD_ALT 4
#define WIN_MENU_MAX_USER 2048 // max menu spec length (= the kernel's
WIN_MENU_MAX)
#define GUI_PTR_Y(v) ((int) (((unsigned long long) (v) & 0xFFFF))
#define GUI_PTR_X(v) ((int) (((unsigned long long) (v) >> 16) & 0xFFFF))
#define GUI_PTR_BUTTONS(v) ((int) (((unsigned long long) (v) >> 32) & 0xFF)) //
held mask
#define GUI_PTR_CHANGED(v) ((int) (((unsigned long long) (v) >> 40) & 0xFF)) // 1
left/2 right/4 mid
#define GUI_PTR_WHEEL(v) ((int) (signed char) (((unsigned long long) (v) >> 48) &
0xFF)) // +fwd/-back
```

Logical key codes (GUI_EVENT_KEY value). Printable keys are their ASCII value.

```
#define KEY_BACKSPACE 8
#define KEY_TAB 9
#define KEY_ENTER 13
#define KEY_UP 0x100
#define KEY_DOWN 0x101
#define KEY_LEFT 0x102
#define KEY_RIGHT 0x103
#define KEY_HOME 0x104
#define KEY_END 0x105
#define KEY_PGUP 0x106
#define KEY_PGDN 0x107
#define KEY_DEL 0x108
#define KEY_F1 0x110 // .. KEY_F12 = 0x11B (KEY_F1 + n - 1)
#define KEY_F12 0x11B
```

windowing

The windows are UIKit's (2026-10-08): `uk_win_create`, `uk_win_present`, `uk_win_menu_set`, `uk_win_list...` in `uikit/win.h` -- the graphics server (Elegant; PocketUI to come) is spoken to by each server's UIKit, its port. The flags, the events and the keys above and the structures of `kern/kapi_abi.h` keep their place here: the events come through the kernel's pump (`kapi_pump_events`, `kapi_pump_wait`, `kapi_should_exit`).

```
int kapi_launch (const char *n);    // start the app n (SD:apps/<n>.app/main) as a
new process -> 1, 0 failure
```

Every task, one line each " " (state R / S / B / N, kind a = an app or k = a kernel task), into b (s bytes) → how many.

```
int kapi_list_tasks (char *b, unsigned s);
int kapi_kill (const char *name);    // kill the app of that name -> 1, 0 (not
running, a kernel task, the caller)
```

ps / kill by PID. `list_procs`: lines " <a|k> ". `kill_pid`: force 0 = clean close, 1 = hard terminate; 1 ok / 0 no such pid / -1 protected.

```
int kapi_list_procs (char *b, unsigned s);
int kapi_kill_pid (int pid, int force);
```

(v91) A process's tree: its descendants (the processes it spawned, the ones they spawned...). `op KAPI_TREE_LIST` → how many descendants pid has (up to cap of their pids into out); `KAPI_TREE_KILL`: pid and all of them terminated now, the leaves first → how many were; `KAPI_TREE_KILL_CHILDREN`: its descendants only. -`KAPI_ESRCH` no such process, -`KAPI_EPERM` a kill that would take the caller, -`KAPI_EINVAL`, -`KAPI_ENOSYS` (a kernel before v91). A terminal closing a tab: its shell and all it runs.

```
int kapi_proc_tree (int pid, int op, int *out, unsigned cap);
```

Keyboard layout: switch among the compiled-in country maps; read the current one.

```
int kapi_set_keymap (const char *name);
int kapi_get_keymap (char *b, unsigned s);
```

Modal-dialog button sets (used by the user-side `ui::MessageBox` in `uidialog.hpp`; the kernel `message_box/file` dialogs were removed -- the toolkit draws its own).

```
#define MB_OK          0
#define MB_OKCANCEL    1
#define MB_YESNO       2
#define MB_YESNOCANCEL 3    // uikit: Yes = 1, No = 2, Cancel / Esc = 0
```

Run an ELF at an absolute path with an argv string (fire-and-forget). 1/0.

```
int kapi_exec (const char *path, const char *args);
```

Framebuffer size in pixels (for edge-pinned borderless windows).

```
void kapi_screen_size (int *w, int *h);
unsigned kapi_get_ticks (void);
void kapi_msleep (unsigned ms);
void kapi_yield (void);
void kapi_exit (int s);
```

(The kernel-drawn widget wrappers -- `kapi_add_/kapi_widget_` -- were removed: every app now builds its UI with the user-side `uikit.hpp` toolkit. See `uikit.hpp`.)

events

Run what is pending -- the kapi_post calls, then this program's events through their handlers -- and return (no wait).

```
void kapi_pump_events (void);  
void kapi_wait_for_exit (void); // pump the events (sleeping between them) until this window is asked to close  
int kapi_should_exit (void); // 1 once this window was asked to close (its close box, uk_win_app_toggle), else 0
```

kernel-font text

Draw kernel-font text into an arbitrary app-mapped 0x00RRGGBB buffer (e.g. a window- chrome copy from uk_win_chrome). Transparent background; dst must be a user VA.

```
void kapi_draw_text_buf (unsigned *dst, int dw, int dh, int x, int y, const char *s, unsigned c);  
int kapi_font_width (void);  
int kapi_font_height (void);
```

Memory snapshot (KB): total RAM, free, app-owned, page size. Any pointer may be 0.

```
int kapi_meminfo (unsigned long *total_kb, unsigned long *free_kb, unsigned long *app_kb, unsigned *page_kb);
```

ABI v33: firmware-detected board RAM + app page-pool (HIGH zone) total/free, the bytes reclaimed above 4GB, and the high-segment count. All KB; any pointer may be 0. (detected = physical board RAM e.g. 8192 MB; apppool = the zone backing app frames via palloc_high.)

```
int kapi_ram_detail (unsigned long *detected_kb, unsigned long *apppool_kb, unsigned long *apppool_free_kb, unsigned long *above4g_kb, unsigned *nsegments);
```

Per-process heap: move the break by inc bytes (Unix sbrk); returns the previous break or (void*)-1. The user allocator (umm.h) is built on this; apps rarely call it.

```
void * kapi_sbrk (long inc);
```

enumeration + clock

The names of the installed apps (the folders SD:apps/.app), one a line, into b (s bytes) → how many.

```
int kapi_list_apps (char *b, unsigned s);
```

The local date and time (any pointer may be 0) → 1 a real date, 0 the clock is not set yet (the time since the boot).

```
int kapi_get_datetime (int *y, int *mo, int *d, int *h, int *mi, int *se);  
int kapi_app_dir (char *b, unsigned s); // "SD:apps/<this process's name>.app/" into b (s bytes) -> its length
```

console + files

b (the first 128 bytes of its n) to the kernel's log as a line of "app" → n, -1 a bad buffer (fd is not used).

```
int kapi_write (int fd, const void *b, unsigned n);
```

Open a file to read it (on the card, the RAM volume or a provider's path, relative to the working directory) → its handle, 0 failure.

```
void * kapi_open (const char *p);  
int kapi_read (void *h, void *b, unsigned n); // up to n bytes from the file's  
position (advanced) -> the bytes read (0: the end), -1 error  
unsigned kapi_fsize (void *h); // the file's size in bytes (0xFFFFFFFF: over 4 GB,  
see kapi_fsize64); 0 for a bad handle  
void kapi_close (void *h); // close a file opened with kapi_open
```

save_file: the whole file written (created / replaced): the bytes written (>= 0; an empty file: 0 -- test < 0 for a failure, or == n), or -1.

```
int kapi_save_file (const char *p, const void *b, unsigned n);
```

Working directory: chdir (1/0) + getcwd. Relative paths in file ops resolve here.

```
int kapi_chdir (const char *p);  
int kapi_getcwd (char *b, unsigned n);
```

directory listing

```
void * kapi_opendir (const char *p); // open a folder to list it -> its handle, 0  
failure (not a folder)  
int kapi_readdir (void *d, struct kapi_dirent *e); // the next entry (name, size,  
is_dir) into *e -> 1, 0 at the end / on error  
void kapi_closedir (void *d); // close a folder opened with kapi_opendir
```

mkdir / remove / rename: 0 = success, -1 = failure (FatFs result).

```
int kapi_mkdir (const char *p);  
int kapi_remove (const char *p);  
int kapi_rename (const char *from, const char *to);
```

stdio / streams / processes

```
void * kapi_pipe (void); // a new pipe (a FIFO in memory) -> its stream handle, 0  
failure  
void * kapi_file_in (const char *p); // a file as a stream to read -> its stream  
handle, 0 failure
```

A file as a stream to write, created / emptied or (append != 0) written at its end → its stream handle, 0 failure.

```
void * kapi_file_out (const char *p, int append);
```

Up to n bytes from a stream (a pipe waits for its writer) → the bytes read, 0 the end / a bad handle.

```
int kapi_stream_read (void *h, void *b, unsigned n);
```

Up to n bytes from a stream without waiting → > 0 the bytes read, 0 the end / a bad handle, -1 nothing yet.

```
int kapi_stream_read_nb (void *h, void *b, unsigned n);
```

n bytes to a stream (a full pipe waits for its reader) → the bytes written, -1 error.

```
int kapi_stream_write (void *h, const void *b, unsigned n);  
void kapi_stream_close (void *h); // close a stream handle (its reference to the  
stream dropped)
```

```
void kapi_stream_eof (void *h); // tell the stream's readers it has ended (the writer is done); the handle stays open
```

1 if the process of kapi_spawn has finished (a bad handle too), 0 if it runs (the handle stays, kapi_wait closes it).

```
int kapi_proc_done (void *proc);  
int kapi_stdin_read (void *b, unsigned n); // up to n bytes from this process's standard input -> the bytes read, 0 the end / none
```

n bytes to this process's standard output (it has none = to the kernel's log, 128 bytes at most) → the bytes written, -1 error.

```
int kapi_stdout_write (const void *b, unsigned n);
```

Start the program at path with the arguments args, its standard input / output the stream handles in / out (0 = none) → its process handle (kapi_wait, kapi_proc_done), 0 failure.

```
void * kapi_spawn (const char *path, const char *args, void *in, void *out);  
int kapi_wait (void *proc); // wait for a spawned process's end -> its exit status (the handle closed), -1 not a process handle  
int kapi_get_args (char *b, unsigned n); // this process's argument string into b (n bytes) -> its length
```

This task's own stdin/stdout stream handles (for a shell wiring children). 0 = none.

```
void * kapi_stdin (void);  
void * kapi_stdout (void);
```

Read the next kernel log event (real-time tee). 1 + fills fields, or 0 if empty.

```
int kapi_klog_read (int *sev, char *src, unsigned sc, char *msg, unsigned mc);
```

Verbose kernel logging: toggle (1/0) at runtime + read the current state.

```
int kapi_set_verbose (int on);  
int kapi_get_verbose (void);
```

TCP/IP sockets over WLAN (ABI v21). net_status: 1 + dotted IP into ip[] if the link is up, else 0. tcp_connect: host = dotted-quad or DNS name; >=0 handle / <0 error. tcp_send: blocking, the bytes queued / <0 -- a short count: a 5 s timeout after those (they are sent; send the rest again). tcp_recv: NON-BLOCKING -- >0 bytes, 0 nothing yet, <0 closed/error. tcp_close: drop the connection.

```
int kapi_net_status (char *ip, unsigned cap);  
int kapi_tcp_connect (const char *host, unsigned port);  
int kapi_tcp_send (int sock, const void *buf, unsigned len);  
int kapi_tcp_recv (int sock, void *buf, unsigned len);  
void kapi_tcp_close (int sock);
```

TCP server side (ABI v37). tcp_listen: listening handle >=0 / <0 (-6 port in use). tcp_accept: BLOCKS until a peer connects; connected handle >=0 + peer IP / <0.

```
int kapi_tcp_listen (unsigned port);  
int kapi_tcp_accept (int listen_sock, char *ip, unsigned cap);
```

Remote screen (ABI v38). screen_grab: composite the screen into dst (w*h 0x00RRGGBB, w/h = kapi_screen_size) → 1 / 0, or 2 = unchanged since the previous grab into the same dst (left as is: skip the diff). inject_pointer/inject_key: input as if from the USB mouse (buttons bit0 L, bit1 R, bit2 M; wheel = notches) / keyboard (cooked string).

```
int kapi_screen_grab (unsigned *dst, int w, int h);
void kapi_inject_pointer (int x, int y, unsigned buttons, int wheel);
void kapi_inject_key (const char *keys);
```

Named IPC services (ABI v40): ipc_register → become service name (1 / 0 taken); ipc_lookup → its pid or 0. Messages go through kapi_mailbox_send / _recv (<= 512 B).

```
int kapi_ipc_register (const char *name);
int kapi_ipc_lookup (const char *name);
```

System clipboard (ABI v40): see clipboard.h for the helpers. Types:

```
#define CLIP_TEXT    1    // plain text
#define CLIP_FILES   2    // file/folder paths, '\n'-separated (copied)
#define CLIP_FILES_CUT 3    // same, cut (the paste moves them)
```

The clipboard replaced by the n bytes of d (64 KB at most, n 0 empties it) of the type CLIP_* → the bytes kept (0 too for a bad pointer, the clipboard then as it was).

```
int kapi_clipboard_set (int type, const void *d, unsigned n);
```

Up to cap bytes of the clipboard into b, its type and its serial (which changes at every set) → the content's whole length, 0 empty. b, type and serial may each be 0.

```
int kapi_clipboard_get (int *type, void *b, unsigned cap, unsigned *serial);
```

End of session (0 = halt, 1 = restart).

```
#define SHUTDOWN_HALT    0
#define SHUTDOWN_RESTART 1
void kapi_shutdown (int mode); // end the session: the card unmounted, then halt
                               (SHUTDOWN_HALT) or restart; does not return
```

Full-screen apps (ABI v41): fullscreen_begin → a screen-sized 0x00RRGGBB back buffer (w/h filled); the desktop stops drawing and all input (screen coords) comes to you. Draw, present_fb to show it; fullscreen_end (or exiting) gives the desktop back. (2026-10-08) The kernel's primitives: a program calls UIKit's uk_win_fullscreen_begin / _end (UIKit/win.h), which tells its graphics server first, then calls these. kapi_fullscreen_begin no longer tells the server.

```
unsigned * kapi_fullscreen_begin (int *w, int *h);
void kapi_present_fb (void);
void kapi_fullscreen_end (void);
```

The keyboard's modifiers (ABI v42; drag and drop is UIKit's: uk_win_drag_begin / data). get_modifiers: MOD* held now; inject_modifiers: set them (vncd).

```
unsigned kapi_get_modifiers (void);
void kapi_inject_modifiers (unsigned mods);
```

Network tools (ABI v43). net_ping: one ICMP echo (RTT in us, or -1 down / -3 unresolved / -4 timeout / -5 send failed). net_resolve: DNS → dotted IP (1/0). net_info: netstat text.

```
int kapi_net_ping (const char *host, unsigned seq, unsigned timeout_ms, char *ip,
unsigned cap);
int kapi_net_resolve (const char *host, char *ip, unsigned cap);
int kapi_net_info (char *buf, unsigned cap);
```

User-space file-system providers (ABI v44, kern/vfs.h): a provider app serves every path starting with its prefix ("FTP:"); the file kapis on those paths reach it as requests.

```

#define VFS_OP_OPEN 1 // path -> status = fid (>= 0); data = u32 file size
#define VFS_OP_READ 2 // a0 fid, a1 offset, a2 length -> data, status = n
#define VFS_OP_CLOSE 3 // a0 fid
#define VFS_OP_LIST 4 // path -> data = (u32 size LE, u8 is_dir, name '\0')*,
status = count
#define VFS_OP_SAVE 5 // path, payload (vfs_req_data) -> status = bytes written
#define VFS_OP_MKDIR 6 // path -> 0 / -1
#define VFS_OP_REMOVE 7 // path -> 0 / -1
#define VFS_OP_RENAME 8 // path -> path2: 0 / -1
int kapi_vfs_register (const char *prefix); // this process serves the paths starting
with prefix -> 1, 0 (taken by another, no room)

```

The next request for this provider into *req → 1, 0 none (with blocking != 0, after waiting up to 0.5 s for one).

```
int kapi_vfs_next (struct kapi_vfs_req *req, int blocking);
```

Up to cap bytes of the request id's payload (VFS_OP_SAVE's data) from offset into buf → the bytes copied, 0 none.

```
int kapi_vfs_req_data (unsigned id, void *buf, unsigned cap, unsigned offset);
```

Answer the request id with its status and len bytes of data (0 = none), its caller woken → 1, 0 (no such request, bad data).

```
int kapi_vfs_reply (unsigned id, int status, const void *data, unsigned len);
```

Wi-Fi scan (ABI v45): the access points around, strongest first (struct kapi_wlan_ap: ssid, bssid, security WLAN_SEC_OPEN/WEP/WPA/WPA2, channel, freq MHz, level dBm, connected). Takes ~3 s (blocks the caller); returns how many (0 = none / no Wi-Fi).

```
int kapi_wlan_scan (struct kapi_wlan_ap *out, int max);
```

Sound (ABI v46), on the 3.5 mm jack. First acquire the output (1 ok, 0 another app has it, -1 no audio); only the owner plays, and its release -- or its exit -- silences it.

```

kapi_sound_start (voice 0..15, milli-Hz (440 Hz = 440000),
SOUND_SQUARE/SINE/TRIANGLE/
SAW/NOISE, volume 0..255): the note plays until kapi_sound_stop
(voice)
(-1 = all). Short attack / release ramps: no clicks.
kapi_sound_write (s16 L/R frames at SOUND_RATE, n): a PCM stream mixed with the
voices;
returns the frames taken (0 = full, retry later) -- audio
players.

```

```
int kapi_sound_acquire (void);
void kapi_sound_release (void);
```

(RETIRED 2026-10-05: the voices are AudioKit's -- ak_fm_start / ak_fm_stop / ak_fm_instrument, audiokit/audiokit.h --; these three answer -1)

```

int kapi_sound_start (int voice, unsigned millihz, int wave, int volume);
int kapi_sound_stop (int voice);
int kapi_sound_write (const short *frames, unsigned n);
int kapi_sound_status (unsigned *rate, unsigned *free_frames, unsigned *owner);

```

FM (ABI v47): give a voice a 2-operator FM instrument (struct kapi_fm_instrument, see kern/kapi_abi.h), then kapi_sound_start (voice, milliHz, SOUND_FM, volume) keys it on and kapi_sound_stop (voice) keys it off (its release rate fades it out).

```
int kapi_sound_instrument (int voice, const struct kapi_fm_instrument *ins);
```

Held keys (ABI v48), for games (key events only report presses): 1 while `key` is held and this window has the keyboard. `key` = KEY_UP/DOWN/LEFT/RIGHT, KEY_ENTER, 27 (Esc), ' ', 'a'..'z' (US position of the key on a USB keyboard), '0'..'9'. Returns 0 on an older kernel.

```
int kapi_key_held (int key);
void kapi_inject_key_held (int key, int down);
```

Run a program under another process name (ABI v49): a runner running an app is named after the app (its window, `list_windows`, `raise_app`). See `launch.h`.

```
int kapi_exec_as (const char *path, const char *args, const char *name);
```

USB gamepads (v50): the raw state of pad 0..3 (1 = there); `user/Include/gamepad.h` maps its buttons.

```
int kapi_pad_state (int index, struct kapi_pad *out);
```

App cores (v51): acquire core 2 or 3, run a function of this app there (no kapi call and no malloc in it: compute, and exchange data through memory), poll its state, release.

```
int kapi_core_acquire (void);
int kapi_core_run (int core, void (*fn) (void *), void *arg, void *stack_top);
int kapi_core_state (int core);
void kapi_core_release (int core);
```

The V3D GPU (v52): `gpu_info` brings it up (1 = usable, `buf` says what / why not); `gpu_draw` renders a depth-tested triangle list (NDC positions + RGBA8 colours) into pixels.

```
int kapi_gpu_info (char *buf, unsigned cap);
int kapi_gpu_draw (const struct kapi_gpu_vertex *v, unsigned n, unsigned clear,
unsigned *pixels, int w, int h, int stride);
```

The GPU's full pipeline (v53): textures (0xAARRGGBB; handle < 0 = a new one, pixels 0 = free), and `gpu_render`: batches (each: its vertices, matrix, texture, blending, depth test, culling) drawn in one frame into `f`→pixels. See `kern/kapi_abi.h`.

```
int kapi_gpu_texture (int handle, const unsigned *pixels, int w, int h, int stride);
int kapi_gpu_render (const struct kapi_gpu_frame *f, const struct kapi_gpu_vertex3
*v, unsigned nv,
const struct kapi_gpu_batch *b, unsigned nb);
```

Full screen straight into the displayed framebuffer (v55; after `kapi_fullscreen_begin`): its pixels (stride in pixels), or 0 (keep drawing into the back buffer + `present_fb`).

```
unsigned * kapi_fullscreen_direct (int *w, int *h, int *stride);
```

The read position of an opened file (v57): 0, or -1 (an older kernel, a file not seekable).

```
int kapi_seek (void *h, unsigned long long pos);
```

Writable + executable memory for generated code, a JIT (v58): 0 on an older kernel / full. After writing code: clean the D-cache / invalidate the I-cache over it (`__builtin__clear_cache`).

```
void * kapi_code_alloc (unsigned long size);
```

A file's whole size (v59: over 4 GB on exFAT; an older kernel: `fsize`).

```
unsigned long long kapi_fsize64 (void *h);
```

(v71) vol_info: a volume's room -- "SD:", "SD1:", "RAM:" (the RAM volume: files in memory until the Pi restarts), a path on it → 0 and *out (total / free / used bytes, its type, KAPI_VOL_RAM), -1 no such volume / an older kernel. (RAM: paths work with every file call above from v71.)

```
int kapi_vol_info (const char *path, struct kapi_vol_info *out);
```

(v93) The volumes: SD:, SD1:..SD3: (the card's partitions), USB1:, USB2:, USB3: (USB sticks and disks whole; USB1P1:, USB1P2:... the partitions of one that has several; USB: is USB1:; mounted when plugged in), RAM:. vol_list: every volume (struct kapi_volume: its state KAPI_VST_, flags KAPI_VF_, sizes, type, label, the files open on it; flags KAPI_VOLS_ROOM: the free space too) → how many there are (out: up to max). A USB volume pulled out stays listed as KAPI_VST_REMOVED (KAPI_VF_UNSAFE if it was mounted) until a device takes its place; its gen changes at each event.

```
int kapi_vol_list (struct kapi_volume *out, int max, unsigned flags);
```

vol_eject: a USB device ("USB1:", or any of its partitions: all of them) made safe to remove -- its written files synced, the device's cache flushed, unmounted → 0; -KAPI_EBUSY files are open on it (synced, still mounted; KAPI_EJECT_FORCE: ejected anyway, they then fail with -KAPI_EIO), -KAPI_EINVAL not a removable volume, -KAPI_ENOENT not mounted.

```
int kapi_vol_eject (const char *vol, unsigned flags);
```

vol_mount: a USB volume ejected but still plugged in (or SD1..SD3) mounted again → 0 / -KAPI_E*.

```
int kapi_vol_mount (const char *vol);
```

vol_format: the volume emptied, a new file system made (struct kapi_format: KAPI_FMT_AUTO / FAT / FAT32 / EXFAT, the cluster, the label) and mounted → 0, -KAPI_EPERM (SD:, never; SD1..SD3 without KAPI_FMT_CARD), -KAPI_EBUSY (files open: KAPI_FMT_FORCE), -KAPI_ENODEV, -KAPI_EINVAL (the label: 11 characters, none of "+,.,/;=<?[]|"), -KAPI_ENOSPC (too small or too big for it), -KAPI_EIO. The caller waits while it runs (seconds on a big stick).

```
int kapi_vol_format (const char *vol, const struct kapi_format *fmt);
```

(v73) The event pump's kernel half -- what kapi_pump_events does, step by step, for a pump of the app's own (a protected app's table runs its pump that way, kern/el0.h). pop_event: the window's next event → 1 (*ev; its handler NOT called), 0 none; event_mods: what kapi_get_modifiers says while a key handler runs (ev→mods), returns the previous value to put back; pop_post: the next kapi_post call → 1 (*p, not run), 0 none; pump_sleep: kapi_pump_wait without the pump (→ how many are pending). An older kernel: 0 / 0xFFFFFFFF / 0 / 0 (no sleep).

```
int kapi_pop_event (struct kapi_event *ev);
unsigned kapi_event_mods (unsigned mods);
int kapi_pop_post (struct kapi_posted *p);
int kapi_pump_sleep (unsigned timeout_ms);
```

(v74) A process's system calls (pid 0: the caller): the total, the rate per second, the 8 table slots most called (user/BinUtils/kapi_names.h names them) → 0, -1 no such process / older kernel, -2 bad pointer.

```
int kapi_proc_stats (int pid, struct kapi_syscall_stats *out);
```

(v75) The POSIX layer's kernel half (docs/POSIX-PLAN.md; the structures and KAPI_* values in kern/kapi_abi.h). Every call returns >= 0 on success, -KAPI_Exxx (newlib's errno value) on failure, and -KAPI_ENOSYS on an older kernel or until the call is implemented. libonxposix (user/Runtime/libc/posix) wraps them as the POSIX functions. v75 WP-MEM: virtual memory

(vm_map = mmap, vm_protect = mprotect, vm_advise = madvise...), threads with their stack size and TLS (struct kapi_thread_attr).

```
long long kapi_vm_map (unsigned long long addr, unsigned long long len, unsigned
prot, unsigned flags);
int kapi_vm_unmap (unsigned long long addr, unsigned long long len);
int kapi_vm_protect (unsigned long long addr, unsigned long long len, unsigned prot);
int kapi_vm_advise (unsigned long long addr, unsigned long long len, int advice);
int kapi_vm_query (unsigned long long addr, struct kapi_vm_region *out);
int kapi_vm_stats (int pid, struct kapi_vm_stats *out);
int kapi_thread_create_ex (const struct kapi_thread_attr *attr);
int kapi_thread_info (int tid, struct kapi_thread_info *out);
```

v75 WP-FILE/PROC: open files with 64-bit offsets (pread / pwrite), stat, unlink / rename of open files, dir_read (255-character names), non-blocking pipe writes; spawn_ex / proc_wait, argv / environment blocks, getpid, clock_info (CNTPTC + UTC sample), sleep_us. file_read / file_write: off -1 = at the handle's offset (advanced; O_APPEND writes at the end), else a pread / pwrite; a write past the end fills the gap with zeros. A file unlinked or renamed while open stays usable through its handles. proc_wait (h, KAPI_WAIT_NOHANG, &st) on a running child → 0 with st.pid set and st.reason -1 (spawn_ex returns once the child has its pid). get_argv / get_env (0, 0) → the block's size. docs/02 §8 "v75: files and processes".

```
long long kapi_file_open (const char *path, unsigned flags, unsigned mode);
long long kapi_file_read (long long h, void *buf, unsigned long long len, long long
off);
long long kapi_file_write (long long h, const void *buf, unsigned long long len, long
long off);
long long kapi_file_seek (long long h, long long off, int whence);
int kapi_file_truncate (long long h, long long size);
int kapi_file_sync (long long h);
int kapi_file_stat (long long h, struct kapi_stat *out);
int kapi_file_close (long long h);
int kapi_path_stat (const char *path, struct kapi_stat *out);
int kapi_path_unlink (const char *path, unsigned flags);
int kapi_path_mkdir (const char *path, unsigned mode);
int kapi_path_rename (const char *from, const char *to);
int kapi_path_utime (const char *path, long long mtime);
int kapi_dir_read (void *dir, struct kapi_dirent2 *out);
int kapi_stream_write_nb (void *h, const void *buf, unsigned len);
long long kapi_spawn_ex (const struct kapi_spawn_attr *a);
int kapi_proc_wait (void *proc, unsigned flags, struct kapi_proc_status *out);
int kapi_get_argv (char *buf, unsigned cap);
int kapi_get_env (char *buf, unsigned cap);
int kapi_getpid (int which);
int kapi_clock_info (struct kapi_clock_info *out);
int kapi_sleep_us (unsigned long long us);
```

v75 WP-NET: BSD sockets (IPv4 TCP / UDP: non-blocking connect, accept, MSG_PEEK...) and poll over sockets, streams and files.

```
int kapi_sock_open (int type, unsigned flags);
int kapi_sock_connect (int s, const struct kapi_sockaddr *to);
int kapi_sock_bind (int s, const struct kapi_sockaddr *addr);
int kapi_sock_listen (int s, int backlog);
int kapi_sock_accept (int s, struct kapi_sockaddr *peer, unsigned flags);
long long kapi_sock_send (int s, const void *buf, unsigned long long len, unsigned
flags, const struct kapi_sockaddr *to);
long long kapi_sock_recv (int s, void *buf, unsigned long long len, unsigned flags,
struct kapi_sockaddr *from);
int kapi_sock_shutdown (int s, int how);
```

```

int kapi_sock_close (int s);
int kapi_sock_getopt (int s, int opt, int *value);
int kapi_sock_setopt (int s, int opt, int value);
int kapi_sock_name (int s, int peer, struct kapi_sockaddr *out);
int kapi_poll (struct kapi_pollfd *fds, unsigned n, int timeout_ms);

```

(v76, WP-IPC: docs/POSIX-PLAN.md §14) Local sockets (sock_pair: STREAM / SEQPACKET / DGRAM; numbers >= KAPI_SOCKET_LOCAL_BASE, served by every sock_* call and poll), messages carrying handles (sock_sendmsg / sock_recvmsg: struct kapi_msghdr, struct kapi_handle_xfer), shared memory objects (shm_create / shm_open / shm_unlink / shm_ctl / shm_map), handle_close, spawn_ex2 (handles given to the child) and get_handles (the child's side). - KAPI_ENOSYS on an older kernel.

```

int kapi_sock_pair (int type, unsigned flags, int *sv);
long long kapi_sock_sendmsg (int s, const struct kapi_msghdr *m, unsigned flags);
long long kapi_sock_recvmsg (int s, struct kapi_msghdr *m, unsigned flags);
long long kapi_shm_create (unsigned long long size, unsigned flags);
long long kapi_shm_open (const char *name, unsigned oflags, unsigned mode);
int kapi_shm_unlink (const char *name);
long long kapi_shm_ctl (long long h, int op, unsigned long long arg);
long long kapi_shm_map (long long h, unsigned long long addr, unsigned long long len,
unsigned prot,
                        unsigned flags, unsigned long long off);
int kapi_handle_close (long long h);
long long kapi_spawn_ex2 (const struct kapi_spawn_attr *a, const struct
kapi_handle_xfer *handles, unsigned n);
int kapi_get_handles (struct kapi_handle_xfer *out, unsigned cap);

```

(v77) Program images (docs/02 section 7): a program is loaded once and shared by its processes; its key is its canonical path (lower case, the volume first). image_preload: loaded ahead and kept (returns at once; a run of that path then reads nothing from the card); image_unload: its pin and its name taken away (freed with its last process); image_list: the live images (path 0), or the one a run of path would map (1 / 0). -KAPI_ENOSYS on an older kernel.

```

int kapi_image_preload (const char *path);
int kapi_image_unload (const char *path);
int kapi_image_list (const char *path, struct kapi_image_info *out, unsigned cap);

```

(v79) What the running kernel is: "key value" lines (name, abi, built, rev, machine, model, ram; keys may be added) → the text's length; -KAPI_ENOSYS (and "") on an older kernel. /bin/uname.

```

int kapi_kernel_info (char *buf, unsigned cap);

```

(v80) The cores: each one's role (KAPI_CORE_*), the microseconds it was busy, an app core's owner → 0; -KAPI_ENOSYS (out zeroed) on an older kernel. Two reads make a load.

```

int kapi_cpu_stats (struct kapi_cpu_stats *out);

```

(v80) The bytes pid's sockets received and sent, its open sockets (pid 0: every process's) → 0; - KAPI_ENOSYS (out zeroed) on an older kernel.

```

int kapi_net_stats (int pid, struct kapi_net_stats *out);

```

(v82) A window resized by its frame (uk_win_resizable): at the release of a drag the pointer handler gets GUI_EVENT_WINRESIZE: GUI_WINRESIZE_X / _Y (the frame's new top left), _W / _H (the client area's new size) of its value; the app applies them. uikit: Root::setResizable.

```

#define GUI_EVENT_WINRESIZE 20
#define GUI_EVENT_TRAY      21 // (v95) the program's icon of the status area: value

```

```
KAPI_TRAY_OPEN (a double click) / _MENU
#define GUI_WINRESIZE_X(v) ((int) (short) ((unsigned long long) (v) >> 48))
#define GUI_WINRESIZE_Y(v) ((int) (short) ((unsigned long long) (v) >> 32))
#define GUI_WINRESIZE_W(v) ((int) (((unsigned long long) (v) >> 16) & 0xFFFF))
#define GUI_WINRESIZE_H(v) ((int) ((unsigned long long) (v) & 0xFFFF))
```

(v83) A shared library (docs/SHARED-LIBS-PLAN.md): "uikit" is SD:/lib/uikit.so, anything with a '/' or a ':' a path → its export table (unsigned version, size; int (*init) (const TLibImports *)); then its entries, mapped in this process until it ends; 0 with *err* = -KAPI_E (-KAPI_ENOTSUP: the library is older than min_version; -KAPI_ENOSYS on an older kernel). Apps do not call this: the library's bind object does, before main (user/Runtime/lib.h).

```
const void * kapi_lib_open (const char *name, unsigned min_version, int *err);
```

(v97) The graphics server's own library under another name (docs/POCKETUI-TECH-STUDY.md section 3): *path* opened and mapped as *kapi_lib_open* does (its version checked against *min_version*), and from now on every *kapi_lib_open* of *alias* -- any process's, a program's bind of "uikit" -- gets it, before a library whose real path is *alias* (PocketUI: *kapi_lib_open_as* ("SD:/lib/pocket/uikit.so", "SD:/lib/uikit.so", *n*, &*err*)). Only the process holding the graphics server's role (call it after KAPI_WS_REGISTER, before KAPI_WS_DISPLAY: every program of the session then finds the alias); both names under SD:/lib/, never SD:/lib/appkit.so. → its export table; 0 with **err* = -KAPI_EPERM (not the server, or a name refused), -KAPI_EBUSY (alias another live process's, or the caller's on another library), -KAPI_EINVAL (*path* = *alias*), *kapi_lib_open*'s errors, -KAPI_ENOSYS on a kernel before v97. Asked again: the same table. The alias lives while the server or a program holds the library: the server ended, it is kept for the same server started again; another server started, it is dropped.

```
const void * kapi_lib_open_as (const char *path, const char *alias, unsigned min_version, int *err);
```

(v84) The sound's output: KAPI_SND_OUT_AUTO / _JACK / _USB / _HDMI (-1: only ask) → what plays now, what is asked and which outputs are there (KAPI_SND_OUT_NOW / _ASKED / _HAS of the result); -KAPI_ENOSYS on an older kernel (the jack only). The Sound applet; kept in SD:/etc/sound.ini.

```
int kapi_sound_output (int out);
```

(v85) the sound's mixer: the programs that play (each has a channel), a channel's volume and mute

```
int kapi_sound_clients (struct kapi_sound_client *out, int max);
int kapi_sound_client_volume (unsigned pid, int volume, int mute);
```

(v73) Is this process protected (ELO, kern/el0.h)? Its table's memcpy is then user code, next to the table, instead of the kernel's.

```
int kapi_is_protected (void);
```

the master volume 0..10 and mute (-1: keep) → volume | 0x100 if muted (older kernel: 10, not muted)

```
int kapi_sound_volume (int volume, int mute);
```

wpa_supplicant.conf read again + DHCP again, no reboot (-1: none / older kernel)

```
int kapi_wlan_reconnect (void);
```

v61: draws with the app's own QPU shaders (user/Libs/v3d/qpu.h builds them): gpu_program makes / replaces / frees (p = 0) a program, gpu_render2 draws batches of them (kapi_abi.h).

```
int kapi_gpu_program (int handle, const struct kapi_gpu_program *p);
int kapi_gpu_render2 (const struct kapi_gpu_frame *f, const float *v, unsigned nv,
unsigned stride,
const struct kapi_gpu_batch2 *b, unsigned nb, const unsigned
*uni, unsigned nuni);
```

(v62) gpu_render3: the batches' vertices where they are (kapi_gpu_batch3: off, stride), x / y framed by the kernel on the way (view: $x' = v0\ x + v1\ w$, $y' = v2\ y + v3\ w$; 0: as they are)

```
int kapi_gpu_render3 (const struct kapi_gpu_frame *f, const float *v, unsigned
nfloats,
const struct kapi_gpu_batch3 *b, unsigned nb, const unsigned
*uni, unsigned nuni, const float *view);
```

(v63) gpu_vbuf: memory the GPU reads too -- gpu_render3 draws vertices there in place (framed in place: view 0 to draw them again; the clipped triangles into the room past nfloats) → 0 none

```
void * kapi_gpu_vbuf (unsigned bytes);
```

(v70) gpu_texture_rect: a rectangle x, y, w x h of a texture's pixels (0xAARRGGBB) replaced, the rest kept → 0, -1 no GPU / older kernel, -2 bad arguments. (Also v70: KAPI_GPU_F_ALPHA, a frame's target keeping its alpha; the GPU compositing service over these: user/Libs/gpucomp/gpucomp.h.)

```
int kapi_gpu_texture_rect (int handle, int x, int y, int w, int h, const unsigned
*pixels, int stride);
```

(v65) the workspaces (virtual desktops): UIKit's uk_win_desk (set, count) → the current desk | the count << 8 | a change counter << 16, read with these:

```
#define KAPI_DESK_CUR(i) ((i) & 0xFF)
#define KAPI_DESK_COUNT(i) (((i) >> 8) & 0xFF)
#define KAPI_DESK_GEN(i) (((unsigned) (i) >> 16) & 0x7FFF)
```

(v66) screen_set: the screen's resolution now (640 × 480 .. 2560 × 1600, w even); every window kept on the screen and sent GUI_EVENT_DISPLAY_RESIZE → 0; -1 out of bounds; -2 not now (a full-screen app...); -3 the firmware refused it (old size kept); -4 an older kernel. Not kept across a reboot (SD:/cmdline.txt width= / height= are).

```
int kapi_screen_set (int w, int h);
```

(v67) Threads: more tasks in this process (its memory, window, files, sockets), preempted like it; the process ends with its main thread (tid 1). Timeouts in ms: 0 = only try, KAPI_WAIT_FOREVER = none. An older kernel: -3 / nothing (a single thread). kapi_thread_create (fn, arg, stack_size (0: 256 KB), name (0: its tid)) → tid >= 2, -1 no memory, -2 too many (32); fn's return value is its exit code. kapi_thread_join → 0 (*code), -1 timeout, -2 no such thread, -3 itself. The GUI belongs to the thread that pumps (the main one): a worker hands its result over with kapi_post (fn, ctx, value), which the pump (kapi_pump_events / kapi_pump_wait / kapi_wait_for_exit) runs on its thread. kapi_pump_wait (ms) sleeps until an event / a post / the close box, then pumps.

```
int kapi_thread_create (int (*fn) (void *), void *arg, unsigned stack_size, const
char *name);
void kapi_thread_exit (int code);
int kapi_thread_join (int tid, unsigned timeout_ms, int *code);
int kapi_thread_self (void);
```

Synchronisation objects (handles > 0; 256 per process). mutex: recursive, released if its owner thread ends; lock → 0, -1 timeout, -2 bad handle. event: manual reset (stays set until reset) or auto (a wait takes it); wait → 0, -1, -2. barrier: count threads meet; wait → 1 for the last one in, 0 the others. kapi_sync_close frees any of them.

```
int kapi_mutex_create (void);
int kapi_mutex_lock (int h, unsigned timeout_ms);
int kapi_mutex_unlock (int h);
int kapi_event_create (int manual_reset, int initial);
int kapi_event_set (int h);
int kapi_event_reset (int h);
int kapi_event_wait (int h, unsigned timeout_ms);
int kapi_barrier_create (unsigned count);
int kapi_barrier_wait (int h);
int kapi_sync_close (int h);
int kapi_post (void (*fn) (void *ctx, long value), void *ctx, long value);
int kapi_pump_wait (unsigned timeout_ms);
```

A user-space lock between this process's threads (the allocators, newlib): an atomic swap, and a yield while another thread holds it -- nothing to create, a zeroed int is free. Not recursive. (Threads all run on core 0; the swap is still an exclusive load / store pair, since the timer may preempt a thread anywhere in its own code.) (The PC builds of the apps -- the desktop simulator, the NetSurf bench -- get the compiler's atomics and no cores.)

The core this code runs on (0: the main one; 2, 3: an app core). The kernel publishes it in TPIDRRO_ELO (v73, kern/el0.h): every app runs at EL0, where MPIDR_EL1 may not be read (the app would be killed) -- never read MPIDR_EL1 in an app.

(on an app core -- kapi_core_run's code, which makes no kapi call -- it spins instead)

```
void kapi_lock (volatile int *l);
void kapi_unlock (volatile int *l);
```

(v68) A futex. kapi_wait_word (addr, expected, timeout_ms) sleeps while *addr == expected → 0 woken / the value differs, 1 timeout (0 ms: only check), -1 a bad address (not 4-byte aligned, unmapped); may wake spuriously: loop on the condition. kapi_wake_word (addr) wakes its sleepers → how many. Works across processes on a shared surface (keyed by the physical address). A word changed by an app core (no kapi there) is noticed at the next 10 ms tick. kapi_thread_priority (tid 0 self / 1 main / >= 2, prio 0 normal / 1 real time / -1 ask) → the previous one: a real-time thread (an audio pump) runs first whenever it is ready, as long as it sleeps before its time slice ends. An older kernel: -1 / -2.

```
int kapi_wait_word (volatile unsigned *addr, unsigned expected, unsigned timeout_ms);
int kapi_wake_word (volatile unsigned *addr);
int kapi_thread_priority (int tid, int prio);
```

(v68) USB MIDI input: class-compliant devices (keyboards, interfaces), found when plugged in, even later. kapi_midi_read (ev, max) takes up to max queued events (struct kapi_midi_event: time_us, cable, status, data1, data2, device, length), oldest first, never waits → how many; one queue for the system (256 events). kapi_midi_devices () → attached.

```
int kapi_midi_read (struct kapi_midi_event *ev, int max);
int kapi_midi_devices (void);
```

(v69) kapi_screen_native (&w, &h): the monitor's own resolution, from its EDID → 1, 0 unknown. kapi_set_timezone (minutes): the local time's offset from UTC at once (-720 .. 840) → 1 ok. WIN_FLAG_FIXED (a window's flags): not movable, no minimise / maximise / close, kept centred.

```
int kapi_screen_native (int *w, int *h);
int kapi_set_timezone (int minutes);
```

The kernel's microsecond clock (CTimer::GetClockTicks: the ARM counter, same formula), the time base of kapi_midi_event.time_us. No kapi call: an app core may read it too.

```
unsigned kapi_clock_us (void);
```

(v68) Low-latency sound, for the sound owner (kapi_sound_acquire). kapi_sound_config (chunk frames 64..1024 (0: 1024), chunks ahead 1..4 (0: 4)) → the latency now in frames ((ahead + 1) x chunk: 1024 x 4 ~116 ms by default, 256 x 2 ~17 ms), -1 not the owner / older kernel. Keep the sound_write stream shallow too (it holds up to 0.5 s): write when kapi_sound_status's free frames show little is queued. The defaults come back when the owner releases the output.

kapi_sound_map () → the mapped PCM ring (struct kapi_sound_ring, kern/kapi_abi.h), mixed until the owner releases the output, or 0. It is plain memory: an app core (kapi_core_run) fills it with kapi_sound_ring_write, which makes no kapi call.

```
int kapi_sound_config (int chunk_frames, int ahead);
struct kapi_sound_ring * kapi_sound_map (void);
```

Frames the ring can take now.

```
unsigned kapi_sound_ring_free (const struct kapi_sound_ring *r);
```

Copy up to n s16 stereo frames into the ring → the frames taken (no kapi call: app cores too).

```
unsigned kapi_sound_ring_write (struct kapi_sound_ring *r, const short *frames,
unsigned n);
```

Reboot the machine (ABI v25). Does not return. Use to apply settings the kernel only reads at boot -- e.g. after wpaconf rewrites SD:/etc/wpa_supplicant.conf.

```
void kapi_reboot (void);
```

1 if a USB keyboard is attached & ready, else 0 (ABI v26). The `keyb` tool polls this at boot before applying a layout (it can run before USB enumeration finishes).

```
int kapi_kbd_ready (void);
```

Load a keyboard layout from a .kmap blob (ABI v27): "OKM1" + u16 rows + u16 cols + table. The kernel copies it; the caller frees data. 1 on success. See /etc/keymaps.

```
int kapi_set_keymap_data (const char *name, const void *data, unsigned len);
```

Hardware RNG (ABI v30): fill buf[len] with random bytes from the Pi's HW RNG. For cryptographic seeding -- the TLS entropy source uses it. Returns bytes written.

```
int kapi_random (void *buf, unsigned len);
```

Shell surfaces (ABI v35): shared 0x00RRGGBB pixel buffers for the activity-shell compositor. The shell creates one sized to a viewport, passes its id to an app; both map it (same physical frames, own VA), the app draws + presents, the shell composites.

```
int kapi_surface_create (int w, int h);
unsigned * kapi_surface_map (int id);
int kapi_surface_size (int id, int *w, int *h);
void kapi_surface_present (int id);
int kapi_surface_destroy (int id);
```

Activity-shell IPC (ABI v35): the kernel routes opaque {from_pid,type,bytes} messages between per-process mailboxes. A user compositor calls kapi_register_shell to become THE shell; apps post to it with kapi_shell_request; the shell replies / pushes async events with kapi_mailbox_send; both drain with kapi_mailbox_recv (fills *from_pid / *type, returns the payload length, or -1 if empty; blocking != 0 waits).

```
int kapi_register_shell (void);
int kapi_shell_request (int type, const void *in, unsigned len);
int kapi_mailbox_send (int target_pid, int type, const void *in, unsigned len);
int kapi_mailbox_recv (int *from_pid, int *type, void *buf, unsigned cap, int blocking);
```

(v89) The graphics server's own door to the kernel (Elegant, SD:/bin/elegant): the display, the raw input, its wait -- KAPI_WS_* (kern/kapi_abi.h) -- and the transport of UIKit's window calls (its port: KAPI_WS_CALL a request, KAPI_WS_KICK "my pixels changed", KAPI_WS_ACTIVE "is there a server"). Not for programs: a program's windows are UIKit's uk_win_* (uikit/win.h), whoever serves them. → >= 0, or -KAPI_Exxx (-KAPI_ENOSYS: a kernel before v89).

```
long kapi_ws_ctl (int op, long a0, long a1, long a2);
```

Memory primitives (ABI v36): the kernel's (Circle's) memset/memcpy/memmove. Real symbols (AppKit's; weak ones in a program that has the bodies inline) so the linker can see them: the freestanding app Makefiles alias the C names onto them (-Wl,--defsym,memset=kapi_memset ...), which resolves the calls GCC emits on its own (array/struct init and copies).

```
void *kapi_memset (void *dst, int c, unsigned long n);
void *kapi_memcpy (void *dst, const void *src, unsigned long n);
void *kapi_memmove (void *dst, const void *src, unsigned long n);
}
#endif
#endif
```

Friendly aliases used by the demos (create_window, present: UIKit's, uikit/win.h).

```
unsigned get_ticks (void);
void msleep (unsigned ms);
void pump_events (void);
int should_exit (void);
```

AppKit's small services (appkit_lib.inc; they were user/applib.h until 2026-10-05)

For every program, with or without a C library.

Strings. ax_strcat: src appended to dst at *pos (advanced), never past cap, always NUL-terminated. ax_app_path: "SD:apps/" (a suffix: ".app/icon.bmp"). ax_streq: 1 = the same. ax_itoa: a signed int in decimal → its length. ax_fmt2: two digits, zero-padded, into d[0], d[1].

```
void ax_strcat (char *dst, int cap, int *pos, const char *src);
void ax_app_path (char *dst, int cap, const char *name, const char *suffix);
int ax_streq (const char *a, const char *b);
int ax_strlen (const char *s);
int ax_itoa (int v, char *b);
void ax_fmt2 (char *d, int v);
```

The console: a string / a line to the standard output.

```
void ax_puts (const char *s);
void ax_putln (const char *s);
```

A minimal .ini reader: [section] headers and key=value lines, ';' or '#' comments, the spaces around trimmed. One file loaded at a time, a program (the store is the program's own); a load replaces the one before. Limits: INI_MAX entries, INI_STRLEN - 1 characters a name or a value, INI_BUFSZ - 1 bytes of file.

```
app_ini_load (filename)    <the program's own folder>/filename -> the number of
entries, -1 absent
app_ini_load_path (path)   any file
app_ini_get (section, key, def), app_ini_get_int (...)  section 0 or "": the keys
before any [section]
app_ini_count (), app_ini_section / app_ini_key / app_ini_value (i)  the entries,
in the file's order
```

```
#define INI_MAX      64
#define INI_STRLEN   64
#define INI_BUFSZ    2048
int app_ini_load_path (const char *path);    // load the .ini file at path (replacing
the one loaded) -> its number of entries, -1 not opened
int app_ini_load (const char *filename);    // the same for <the program's own
folder>/filename (kapi_app_dir)
```

The value of key in section (0 or "" = the keys before any [section]) of the loaded file, else def.

```
const char *app_ini_get (const char *section, const char *key, const char *def);
```

The value of key in section as a decimal integer (a sign allowed), def when the key is absent or has no digit.

```
int app_ini_get_int (const char *section, const char *key, int def);
int app_ini_count (void);    // the number of entries (key=value lines) of the loaded
file
const char *app_ini_section (int i);    // the section of the entry i (0-based, in
the file's order); "" out of range
const char *app_ini_key (int i);    // the key of the entry i; "" out of range
const char *app_ini_value (int i);    // the value of the entry i; "" out of range
```

The keyboard layout ("FR", "BE"...): SD:/etc/keymaps/.kmap (kapi_set_keymap_data), else a map the kernel has (kapi_set_keymap). Non-zero = done. The keyboard must be up (kapi_kbd_ready).

```
int ax_load_keymap (const char *name);
```

Starting programs (it was user/launch.h until 2026-10-05)

launch.h -- starting programs from user space. The kernel only loads ELF programs; the other formats have a RUNNER (SD:/etc/runners.ini, "extension = program"): a BASIC program (.bas, .bax) is run by SD:/bin/basic -- or an app that says it opens them: an emulator's app.txt names the games it plays, "games = Game Boy Color: gbc; Game Boy: gb" (and "opens = dol" for files it opens that are not games): installing an emulator's package is enough.

```
lx_launch (name, args)    an app: SD:apps/<name>.app/main (an ELF), else its
main.<ext>
lx_open (path, args)      run by that extension's runner (named after the app)
lx_runner (path, out)     a program file: an ELF as it is, else by its runner
                          the runner of a file (by its extension), 0 if none
```

C and C++ (run, init, the apps).

```
#define LX_INI    "SD:/etc/runners.ini"
char lx_low (char c);    // c in lower case ('A'..'Z' only)
int lx_len (const char *s);    // the length of s (0 for a null pointer)
```

```
void lx_cat (char *d, int cap, int *n, const char *s); // s appended to d at *n
(advanced), never past cap, NUL-terminated
int lx_exists (const char *path); // 1 if the file at path can be opened
(kapi_open), else 0
```

The i-th "ext = program" line of runners.ini (0-based): 1 found, 0 past the end.

```
int lx_entry (int i, char *ext, int ecap, char *prog, int pcap);
```

Does the value of an app.txt's "games" / "opens" (the extensions after each "System:", or all the words of "opens") hold ext?

```
int lx_lists_ext (const char *v, int len, const char *ext, int games);
```

The app (SD:/apps/.app/main) whose app.txt opens files of this extension: 1 + out, 0 if none.

```
int lx_app_for (const char *ext, char *out, int cap);
```

The runner of a file, by its extension (runners.ini, else the built-in list): 1 + out, 0 if none.

```
int lx_runner (const char *path, char *out, int cap);
```

The runner's command line: "path" args (the path quoted when it has spaces).

```
void lx_cmdline (char *out, int cap, const char *path, const char *args);
```

A program file: by its runner if its extension has one, else as an ELF. name: the process name (0: the kernel's, from the path). 1 = started.

```
int lx_open_as (const char *path, const char *args, const char *name);
int lx_open (const char *path, const char *args);
```

An app bundle by folder path (".../x.app"): its main (ELF), else main. for a runner.

```
int lx_launch_dir (const char *dir, const char *name, const char *args);
```

An app by name (SD:apps/.app), with arguments (0 / "": none). 1 = started.

```
int lx_launch (const char *name, const char *args);
```

(the tests of the kernel's table, and the PC builds: the bodies inline -- see this header's top)